

相关PPT



视频链接：

<https://meeting.tencent.com/v2/cloud-record/share?id=761bdbd7-dba1-4d33-b2b4-4b085f9e10ec&from=3>

1 01 登录客户服务器

1.1 01.1 根据客户提供的服务器登录信息进行登录服务器操作

大部分客户的服务器登录都需要登录vpn还要堡垒机都各种安全软件，这个时候需要根据客户提供的相关信息进行登录操作。

如果客户服务器有堡垒机的话，登录环境可能需要直接使用堡垒机工具进行登录；如果不是，那可以通过一些ssh工具进行登录，比如putty、xshell、terminus等软件，如果有需要可以找部署老师索要。

1.2 01.2 文件上传到服务器

一般文件上传都可以使用Filezilla工具，无论客户是否使用堡垒机，还是直接通过ssh或者ftp工具。具体使用步骤，可以查看上面的ppt。

2 02 在服务器上进行简单的Linux命令操作

2.1 02.1 ls命令

02.1.1 ls命令的含义

list directory contents

显示指定工作目录下的内容（列出目前工作目录所含的文件及子目录）

实例1：当前实例，ls命令没有加参数，ls命令执行之后，输出的是当前目录下的文件及子目录。

```
[17:15:22] 519 root@ecs-dbf7:~#
[17:15:23] 520 root@ecs-dbf7:~# ls
[17:15:23] 521 install_ubuntu22.04.6.sh packages ubuntu_22.04.6_ubuntu_6.7.0_offline.tar.gz
[17:15:23] 522 install_base_ubuntu22.04.6.sh requirements.txt ubuntu_22.04.6_ubuntu_install.tar.gz
[17:15:23] 523 libssl1.1.1-1ubuntu2.22_amd64.deb snap ubuntu_22.04.6_base_install.tar.gz
[17:15:23] 524 node-local-1.16.0.tar tcpdump_4.99.1-3ubuntu0.1_amd64.deb ubuntu_22.04.6_python3_pip_offline.tar.gz
[17:15:23] 525 root@ecs-dbf7:~#
```

1 ls命令，实例1

实例2：当前实例，ls命令后面添加指定目录，这样可以显示指定目录下的文件及子目录

```
root@ecs-dbf7:~# ls /home/user04/
Desktop Documents Downloads Music Pictures Public Templates Videos
root@ecs-dbf7:~#
```

2 ls命令示例2

ls是linux的基本命令，后面/home/user04/是命令ls的参数。参数和命令ls本身之间是需要有空格的，如果没有空格，linux系统就会识别成为一个命令，比如下图所示

```
Desktop Documents Downloads Music Pictures Public Templates Videos
root@ecs-dbf7:~# ls/home/user04/
-bash: ls/home/user04/: No such file or directory
root@ecs-dbf7:~#
```

3 错误示例3

如上例所示，由于命令ls与后面的参数之间没有添加必要的空格，导致命令执行报错，可以根据实际的报错内容进行对应的调整，比如说给ls与指定目录/home/user04之间添加必须的空格，如：

```
ls /home/user04
```

02.1.2 ls命令语法规则

ls命令格式

ls命令语法

```
ls [选项] [路径]
# ls常用选项
```

- a 含义: all所有, 显示指定目录下所有子目录与文件, 包含隐藏文件
- l 含义: 这个参数是小写的L, 不是数字1, 以列表方式显示文件的详细信息
- h 含义: 配合 -l 以人性化的方式显示文件大小 (文件大小 + 单位)

ls命令加选项-a示例: 对照上述的讲解进行理解。

```
[root@test-k8s-02 ~]# ls -a
. . . .bash_history .bash_logout .bash_profile .bashrc .cache .cshrc .history .kube kubernetes.conf .pki .ssh .tcshrc .viminfo
[root@test-k8s-02 ~]# ls -a /root
. . . .bash_history .bash_logout .bash_profile .bashrc .cache .cshrc .history .kube kubernetes.conf .pki .ssh .tcshrc .viminfo
[root@test-k8s-02 ~]#
```

4 ls -a 示例

ls命令加选项-l示例:

```
[root@test-k8s-02 ~]# ls -l
total 4
-rw-r--r-- 1 root root 481 Mar 27 10:46 kubernetes.conf
[root@test-k8s-02 ~]# ls -l /root
total 4
-rw-r--r-- 1 root root 481 Mar 27 10:46 kubernetes.conf
[root@test-k8s-02 ~]#
```

5 ls -l 示例

ls命令加选项-h示例: (pwd命令作用: 显示当前所在文件夹的绝对路径)

```
16 [root@test-k8s-02 ~]# ls -h
17 kubernetes.conf
18 [root@test-k8s-02 ~]# ls -h /root
19 kubernetes.conf
20 [root@test-k8s-02 ~]# pwd
21 /root
22 [root@test-k8s-02 ~]#
```

2.2 02.2 mkdir命令

2.2.1 02.2.1 mkdir命令使用

mkdir 含义: make directories

描述: Create the DIRECTORY(ies), if they do not already exist. (意思: 如果目录不存在, mkdir命令就会创建目录。如果已经存在, 就不会创建, 会给出已经存在的提示)

```

537 root@ecs-dbf7:~# ls -lh
538 total 314M
539 -rw-r--r-- 1 root root 123 Jun 7 2023 install_ubuntu22.04.6.sh
540 -rw-r--r-- 1 root root 555 Jun 7 2023 install_base_ubuntu22.04.6.sh
541 -rw-r--r-- 1 root root 1.3M Mar 7 10:35 libssl1.1.1-1ubuntu2.22_amd64.deb
542 -rw-r--r-- 1 root root 118M Mar 1 11:39 node-local-1.16.0.tar
543 drwxr-xr-x 2 root root 4.0K Jun 7 2023 packages
544 -rw-r--r-- 1 root root 16 Jun 7 2023 requirements.txt
545 drwx----- 4 root root 4.0K Feb 19 17:11 snap
546 -rw-r--r-- 1 root root 490K Mar 7 11:25 tcpdump_4.99.1-3ubuntu0.1_amd64.deb
547 -rw-r--r-- 1 root root 46M Jun 7 2023 ubuntu_22.04.6_ansible_6.7.0_offline.tar.gz
548 -rw-r--r-- 1 root root 46M Mar 7 10:24 ubuntu_22.04.6_ansible_install.tar.gz
549 -rw-r--r-- 1 root root 53M Mar 7 10:24 ubuntu_22.04.6_base_install.tar.gz
550 -rw-r--r-- 1 root root 53M Jun 7 2023 ubuntu_22.04.6_python3_pip_offline.tar.gz
551 root@ecs-dbf7:~# mkdir packages
552 mkdir: cannot create directory 'packages': File exists
553 root@ecs-dbf7:~#
  
```

6 mkdir命令错误示例

上图示例所示：

首先，使用命令ls -lh 查看当前目录下面的所有内容，内容显示标红部分，已经存在packages目录；

然后，使用命令mkdir 加上参数（即要创建的目录名字）packages，即 mkdir packages。执行该命令之后，有如图所示的提示：cannot create directory 'packages': File exists。意思就是说：你想要创建的目录已经存在，不允许创建，也就是你mkdir packages命令执行是失败的。

2.2.2 02.2.2 mkdir命令语法格式

mkdir命令语法

- 1 mkdir [-p] dirName
- 2 # 选项（选项带有大括号，大括号代表可有可无）
- 3 -p 含义：确保目录名称存在，不存在的就会创建一个（不建议初学者使用）

mkdir命令示例1：

```

85 root@ecs-dbf7:~# ls
86 install_ubuntu22.04.6.sh packages ubuntu_22.04.6_ansible_6.7.0_offline.tar.gz
87 install_base_ubuntu22.04.6.sh requirements.txt ubuntu_22.04.6_ansible_install.tar.gz
88 libssl1.1.1-1ubuntu2.22_amd64.deb snap ubuntu_22.04.6_base_install.tar.gz
89 node-local-1.16.0.tar tcpdump_4.99.1-3ubuntu0.1_amd64.deb ubuntu_22.04.6_python3_pip_offline.tar.gz
90 root@ecs-dbf7:~#
91 root@ecs-dbf7:~# mkdir test
92 root@ecs-dbf7:~# ls
93 install_ubuntu22.04.6.sh requirements.txt ubuntu_22.04.6_ansible_install.tar.gz
94 install_base_ubuntu22.04.6.sh snap ubuntu_22.04.6_base_install.tar.gz
95 libssl1.1.1-1ubuntu2.22_amd64.deb tcpdump_4.99.1-3ubuntu0.1_amd64.deb ubuntu_22.04.6_python3_pip_offline.tar.gz
96 node-local-1.16.0.tar test
97 packages ubuntu_22.04.6_ansible_6.7.0_offline.tar.gz
  
```

7 mkdir test命令示例

在使用mkdir命令之前，查看一下当前目录下面所有的文件以及文件夹情况；然后使用命令mkdir创建一个名字为test的目录进行测试；创建完成之后，通过ls命令查看当前目录，发现新创建的test目录已经存在了。

mkdir命令加选项-p示例2

```

92 root@ecs-dbf7:~# ls
93 install_ubuntu22.04.6.sh requirements.txt ubuntu_22.04.6_ansible_install.tar.gz
94 install_base_ubuntu22.04.6.sh snap ubuntu_22.04.6_base_install.tar.gz
95 libssl1.1_1.1.1f-1ubuntu2.22_amd64.deb tcpdump_4.99.1-3ubuntu0.1_amd64.deb ubuntu_22.04.6_python3_pip_offline.tar.gz
96 node-local-1.16.0.tar test ubuntu_22.04.6_ansible_6.7.0_offline.tar.gz
97 packages
98 root@ecs-dbf7:~# mkdir -p test_01/sub_test
99 root@ecs-dbf7:~# ls
100 install_ubuntu22.04.6.sh requirements.txt ubuntu_22.04.6_ansible_6.7.0_offline.tar.gz
101 install_base_ubuntu22.04.6.sh snap ubuntu_22.04.6_ansible_install.tar.gz
102 libssl1.1_1.1.1f-1ubuntu2.22_amd64.deb tcpdump_4.99.1-3ubuntu0.1_amd64.deb ubuntu_22.04.6_base_install.tar.gz
103 node-local-1.16.0.tar test ubuntu_22.04.6_python3_pip_offline.tar.gz
104 packages test_01
105 root@ecs-dbf7:~# mkdir test_02/sub_test
106 mkdir: cannot create directory 'test_02/sub_test': No such file or directory

```

8 mkdir加选项-p示例2

示例中命令：`mkdir -p test_01/sub_test`，本条命令的作用是创建两层目录或者叫创建两层文件夹，由于test_01文件夹不存在，所以会先创建test_01，然后在test_01中创建sub_test文件夹。

再看示例2命令：`mkdir test_02/sub_test`，本条命令没有加选项-p，由于test_02文件夹不存在，想要在这个文件夹下面创建sub_test文件夹，结果就是如图所示的报错情况。

综上，-p参数的作用一目了然，但是能不用的话，尽量不要使用，除非对Linux命令很熟，知道自己要干啥。

2.3 02.3 cd命令

2.3.1 02.3.1 cd命令使用

cd命令 含义：Change the shell working directory. 意思就是：切换当前工作目录

描述：Change the current directory to DIR. The default DIR is the value of the HOME shell variable.

cd命令示例：

```

683 root@ecs-dbf7:~#
684 root@ecs-dbf7:~# pwd
685 /root
686 root@ecs-dbf7:~# cd /home/user04/
687 root@ecs-dbf7:/home/user04# ls
688 Desktop Documents Downloads Music Pictures Public Templates Videos
689 root@ecs-dbf7:/home/user04# cd
690 root@ecs-dbf7:~#

```

9 cd命令示例

1. 在使用cd命令切换目录之前，我先用命令pwd查看一下当前目录路径为：/root
2. 然后使用cd命令切换到指定目录 /home/user04/，即命令 `cd /home/user04`；切换完成之后你会发现#号之前的路径由~变成了你所切换的目录路径/home/user04
3. 执行ls命令，查看当前目录下面的内容，确实为目录/home/user04下面的内容，其实这里也可以在执行一遍pwd查看当前路径是否为我们所切换的路径（当然第二步的说明足以证明我们是切换成功的，如果不成功，会报错目录不存在的）
4. 当我们执行cd命令不添加任何参数的时候，会直接回到当前用户的家目录。因为我们当前用户是root，而root用户的家目录就是/root，所以我们只执行cd，则目录会回到/root（家目录的缩写为~），具体可以看下图示例

cd命令示例2

```

689 root@ecs-dbf7:/home/user04# cd
690 root@ecs-dbf7:~# su user04
691 user04@ecs-dbf7:/root$ cd
692 user04@ecs-dbf7:~$ pwd
693 /home/user04
694 user04@ecs-dbf7:~$

```

1. 由于上个示例的用户为root用户，而root用户的家目录为/root。为了方便理解家目录的含义。本示例进行简单的讲解
2. 首先，我使用su命令来切换用户（初学者可以不用知道su的用法，只需要知道这个命令是用来切换用户的即可），命令：su user04，命令解释：su是命令，user04是我需要切换的用户名
3. 执行了su user04命令之后，你会发现前面的用户名由root@ecs-dbf7变成了user04@ecs-dbf7，这就说明我们已经由用户root切换到了用户user04。那么@符合后面的ecs-dbf7是什么意思呢？这个其实就是主机名称，你可以理解为这台服务器的名字。
4. 用户切换完成之后，我们执行命令：cd，该命令后面不添加任何参数。如上个例子所示，如果cd命令后面不添加任何参数，那么工作目录就会切换到当前用户的家目录下面。我们执行了cd命令之后，然后再执行一遍pwd命令，发现当前目录已经变成了/home/user04；同时用户名@主机名:后面的路径也变成了~符号，即user04@ecs-dbf7:~，此时的~符合代表的是user04用户的家目录，即/home/user04。而上个例子的家目录是/root，也就说root用户的家目录是/root。

2.3.2 02.3.2 cd命令语法格式

cd命令英文全拼：change directory，即用于改变当前工作目录的命令，切换到指定的路径。cd命令的作用是切换到指定目录，不能到文件！！

此处需要讲三个特殊的字符：

- ~ 表示当前用户的家目录的意思，如果不清楚具体指代的是啥，可以使用pwd目录查看绝对路径
- . 表示当前所在的目录
- .. 表示当前所在目录的上一层目录

```

cd [dirName]
# dirName: 要切换的目标目录，可以是相对路径或绝对路径
# 切换到绝对路径，绝对路径是已/开头的一个完整的路径。/代表是文件系统的根，相当于windows的我的电脑，
# 我的电脑下面又有c盘、d盘等等，就差不多是这样子：/我的电脑/c盘/xxx/xxx/
cd /path/to/directory
# 切换到相对路径，所谓相对路径，一般没有特殊说明，就是相对于你当前所在的路径，如果不知道自己当前所在
# 路径的具体位置，可以使用pwd命令查看。
cd packages/sub_path/sub_sub_path
# 假设当前路径下面有文件夹名字为packages，那么上面命令的意思就是切换目录到packages下面的sub_path
# 下面的sub_sub_path目录下面

# 如果我们想使用cd命令切换到父一层目录同级的一个目录下面的下面的目录，需要怎么操作呢
cd ../parent_brother/parent_brother_son

```

cd命令示例1

```
786 root@ecs-dbf7:~# pwd
787 /root
788
789 root@ecs-dbf7:~# ls
790 install_ubuntu22.04.6.sh      packages      test
791 install_base_ubuntu22.04.6.sh requirements.txt test_01
792 libssl1.1_1.1.1f-1ubuntu2.22_amd64.deb snap          ubuntu_22.04.6_ubuntu_22.04.6_base_install.tar.gz
793 node-localize-1.16.0.tar      tcpdump_4.99.1-3ubuntu0.1_amd64.deb ubuntu_22.04.6_python3_pip_offline.tar.gz
794
795 root@ecs-dbf7:~# cd packages/
796 root@ecs-dbf7:~/packages# pwd
797 /root/packages
798
799 root@ecs-dbf7:~/packages# ls
800 ansible-6.7.0-py3-none-any.whl
801 ansible-core-2.13.10-py3-none-any.whl
802 cffi-1.15.1-cp38-cp38-manylinux_2_17_x86_64.manylinux2014_x86_64.whl
803 cryptography-41.0.1-cp37-abi3-manylinux_2_17_x86_64.manylinux2014_x86_64.whl
804 Jinja2-3.1.2-py3-none-any.whl
805 MarkupSafe-2.1.3-cp38-cp38-manylinux_2_17_x86_64.manylinux2014_x86_64.whl
806 netaddr-0.8.0-py2.py3-none-any.whl
807 packaging-23.1-py3-none-any.whl
808 pycparser-2.21-py2.py3-none-any.whl
809 PyYAML-6.0-cp38-cp38-manylinux_2_5_x86_64.manylinux1_x86_64.manylinux2_12_x86_64.manylinux2010_x86_64.whl
810 resolvelib-0.8.1-py2.py3-none-any.whl
811
812 root@ecs-dbf7:~/packages#
```

10 cd命令使用相对路径

本示例使用的是相对路径进行的目录切换操作：

1. 首先，使用pwd命令看你一下我当前所在的目录的位置，为/root
2. 其次，使用ls命令查看一下我当前目录下面有哪些文件夹和文件
3. 我知道当前目录下面有一个文件夹名为packages，我想要切换到这个目录下面，那么执行命令：`cd packages/`
4. 执行完第三步的命令之后，执行命令pwd，发现当前路径变成了 /root/packages
5. 使用ls命令，查看packages目录下面有哪些内容

cd命令示例2

```
737 root@ecs-dbf7:~/packages# pwd
738 /root/packages
739
740 root@ecs-dbf7:~/packages# cd /root/packages/
741 /root/packages
742 root@ecs-dbf7:~/packages#
```

11 cd命令使用绝对路径操作

本示例使用绝对路径进行目录切换操作

1. 在执行cd命令执行，我们先查看一下当前所在目录的位置，即当前路径为 /root/packages
2. 执行cd命令：`cd /root/packages`，命令执行完成之后，继续使用pwd命令查看当前路径，发现路径仍然是/root/packages
3. 所以，无论使用相对路径进行目录切换还是使用绝对路径切换最终结果都是一样的。

cd命令示例3（可以只了解即可）

```
742 root@ecs-dbf7:~/packages# cd ../packages/
743 root@ecs-dbf7:~/packages# pwd
744 /root/packages
745 root@ecs-dbf7:~/packages#
```

12 cd命令使用特殊符号切换目录

本示例中使用了上面讲到的一个字符：`..`，这俩点代表的是当前目录的上一级目录，不需要写上一级目录的具体名字，只需要用俩点代替即可

1. 使用命令：`cd ../packages`，进行目录切换，切换完成之后使用pwd命令查看当前路径
2. 上述命令和cd /root/packages命令的操作结果是一样的。

2.4 02.4 vim命令（该命令操作可能相对于其他命令来说稍微有点复杂，选学）

2.5 02.5 ping命令

2.5.1 02.5.1 ping命令使用

ping命令 含义：send ICMP ECHO_REQUEST to network hosts（ICMP缩写Internet Control Message Protocol）。即ping命令会使用ICMP传输协议，发出要求回应的西悉尼，若远端主机的网络功能没有问题，就会回应该信息，因而得知该主机运作正常。

ping命令其实很复杂，但是我们只讲最简单的使用，也是对我们最有用的使用方法

ping命令执行之后，会有两种结果：一个结果是对方有返回，另外一个请求不到对方。

ping命令示例1

```

784 root@192:~# ping www.baidu.com
785 PING www.a.shifen.com (110.242.68.3) 56(84) bytes of data.
786 64 bytes from 110.242.68.3 (110.242.68.3): icmp_seq=1 ttl=47 time=11.1 ms
787 64 bytes from 110.242.68.3 (110.242.68.3): icmp_seq=2 ttl=47 time=11.0 ms
788 64 bytes from 110.242.68.3 (110.242.68.3): icmp_seq=3 ttl=47 time=11.0 ms
789 64 bytes from 110.242.68.3 (110.242.68.3): icmp_seq=4 ttl=47 time=11.0 ms
790 ^C
791 --- www.a.shifen.com ping statistics ---
792 4 packets transmitted, 4 received, 0% packet loss, time 3005ms
793 rtt min/avg/max/mdev = 11.003/11.023/11.050/0.016 ms
794 root@192:~# ping 110.242.68.3
795 PING 110.242.68.3 (110.242.68.3) 56(84) bytes of data.
796 64 bytes from 110.242.68.3: icmp_seq=1 ttl=47 time=11.1 ms
797 64 bytes from 110.242.68.3: icmp_seq=2 ttl=47 time=11.0 ms
798 64 bytes from 110.242.68.3: icmp_seq=3 ttl=47 time=11.0 ms
799 64 bytes from 110.242.68.3: icmp_seq=4 ttl=47 time=11.1 ms
800 ^C
801 --- 110.242.68.3 ping statistics ---
802 4 packets transmitted, 4 received, 0% packet loss, time 3003ms
803 rtt min/avg/max/mdev = 11.021/11.062/11.109/0.032 ms
804 root@192:~#

```

13 ping命令可达示例1

本示例说明的是ping命令成功的情况。ping命令后面跟的既可以是域名也可以是ip地址

1. 首先，我们使用百度的网址作为ping的对象，即：ping www.baidu.com¹（注意域名是不包含https://的域名，纯域名，不包含http协议）
2. 命令执行之后，如果是如上例所示通的状态的话，会一直请求不会终端，这个时候需要我们使用ctrl+c命令，进行强制中断请求
3. ping命令被ctrl+C中断之后，后面会输出一个总结性的描述：4 packets transmitted, 4 received, 0% packet loss, time 3005ms，这个描述的大概意思就是4个包进行了传输，接收到了4个包，0个丢失，丢失率为0，time表示的是平均耗时。
4. 然后，我们在使用ping命令，ping一下百度域名对应的IP地址，即：ping 110.242.68.3
5. 命令执行之后，发现也是通的。4 packets transmitted, 4 received, 0% packet loss, time 3003ms

下面示例，我们来看一下不通的情况如何

1. <http://www.baidu.com>

```
72 root@ecs-dbf7:~# ping www.baidu.com
73 PING www.a.shifen.com (110.242.68.4) 56(84) bytes of data.
74
75
76 ^C
77 --- www.a.shifen.com ping statistics ---
78 78 packets transmitted, 0 received, 100% packet loss, time 78841ms
79
80 root@ecs-dbf7:~# ping 110.242.68.4
81 PING 110.242.68.4 (110.242.68.4) 56(84) bytes of data.
82
83 ^C
84 --- 110.242.68.4 ping statistics ---
85 17 packets transmitted, 0 received, 100% packet loss, time 16386ms
86
87 root@ecs-dbf7:~#
```

14 ping命令不通的情况示例

本示例ping命令执行之后，长时间没有任何返回。这个时候只能使用ctrl+C命令进行强制终端，中断之后，会有一个提示信息：78 packets transmitted, 0 received, 100% packet loss, time 78841ms。根据提示信息，我们可以知道有78个网络包发送，但是100% packet loss，也就是说所有包都被丢失了，没有任何响应。其实，意思就是网络是不通的。

在windows上进行测试ping不通的情况

```
C:\Users\lijingliang>ping 8.8.8.8
正在 Ping 8.8.8.8 具有 32 字节的数据:
请求超时。
请求超时。
请求超时。
请求超时。
8.8.8.8 的 Ping 统计信息:
    数据包: 已发送 = 4, 已接收 = 0, 丢失 = 4 (100% 丢失),
C:\Users\lijingliang>
```

15 Windows上测试ping命令示例

Windows对于ping命令设置是比较友好的，如上图所示的提示：请求超时，同时丢失=4(100%丢失)。失败情况一目了然。

2.5.2 02.5.2 ping命令语法

ping命令语法

```
ping [主机名称或IP地址]
# 比如 ping www.baidu.com
# 就是测试本机是否能与域名www.baidu.com可以连通
```

```
ping www.baidu.com
```

2.6 02.6 telnet命令

2.6.1 02.6.1 telnet命令使用

Linux上使用

telnet命令示例1

```
104 root@192:~# telnet www.baidu.com 443
105 Trying 110.242.68.3...
106 Connected to www.a.shifen.com.
107 Escape character is '^]'.
108
```

16 telnet命令测试端口是否通

注意: telnet通了之后, 如何退出这个telnet状态呢? 有一些情况下, 直接输入ctrl+C即可; 有时候可能需要输入quit单词才可以退出。如果一直不成功, 可以多次尝试, 直到可以退出位置。

本实例所示的telnet命令执行的是测试百度的443端口是否是通的, 即:

```
# telnet命令的格式: telnet 主机IP或域名 端口
# 本示例域名为: www.baidu.com , 端口为: 443
telnet www.baidu.com 443
```

通过示例, 可以看到下面的提示为:

```
Trying 110.242.68.3...
Connected to www.a.shifen.com2.
Escape character is '^]'.
```

上述三行输出, 我们来一行行解读。

1. 由于我们输入的是百度的域名, 但是第一行输出为啥是: Trying 110.242.68.3...。这是因为, 所有的域名经过dns解析之后都是有对应的IP地址, dns会分配一个离我们最近的IP地址(即一个域名可以对应多个IP地址)。Trying的意思就是尝试去链接到这个地址(110.242.68.3)上去;
2. 第二行, 我们可以看到: Connected to www.a.shifen.com³, 说明已经链接到了百度的一个子域名上去了。[后面这些可以不用看]提示: Connected to, www.a.shifen.com是百度域名的一个别名。
3. Escape character is '^]' 这个只是一个连通之后的提示, 看到这个提示说明已经连通了。

Windows上使用

每台windows上面的提示可能会不太一样, 有的会跟Linux一样的提示, 但是我这边的提示可能不太一样, 具体如下图所示:

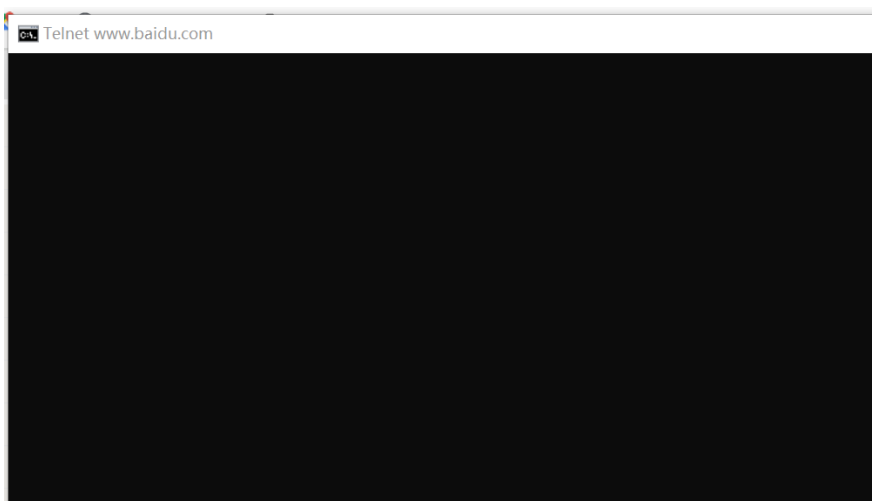
-
2. http://www.a.shifen.com
 3. http://www.a.shifen.com



```
C:\windows\system32\cmd.exe
C:\Users\lijingliang>
C:\Users\lijingliang>
C:\Users\lijingliang>
C:\Users\lijingliang>telnet www.baidu.com 443
```

17 这里只是输入了telnet命令，但是并没有回车执行

接下来，我们回车执行telnet www.baidu.com⁴ 443这条命令



```
Telnet www.baidu.com
```

18 telnet www.baidu.com 443执行之后结果

命令执行之后，会有一个空白的界面，这个时候代表是通的。退出的话，可以执行Ctrl+C或者输入quit（输入的时候不会显示，但是只要保证输入正确，直接回车即可）

2.6.2 02.6.2 telnet命令语法

telnet命令含义：user interface to the telnet protocol. 意思就是用telnet命令登录远端服务器

描述：The telnet command is used for interactive communication with another host using the TELNET protocol.

telnet命令使用，需要明确主机域名或者主机IP，然后需要知道指定的端口号。

4. <http://www.baidu.com>

2.7 02.7 curl命令

2.7.1 02.7.1 curl命令使用

curl命令是一个利用URL规则在命令行下规则的文件传输工具。我们一般用该命令来测试一个api是不是通。

curl命令示例1:

```
[root@hr-pro-worker-10 document]# curl http://api.beisenappaa.com/Resume/v2/101440/applicant/summary/resume
{
  "StatusCode": 200,
  "Message": "Resource not found",
  "ErrorCode": "406",
  "AdditionalDetails": "",
  "Version": "None",
  "Headers": [],
  "Properties": {
    "httpResponse": {
      "Headers": [],
      "StatusCode": 200,
      "StatusDescription": null,
      "SuppressEntityBody": false,
      "SuppressPreamble": false
    }
  },
  "IsFault": false,
  "IsEmpty": false,
  "State": 0
}[root@hr-pro-worker-10 document]#
```

19 curl命令示例1

此处仅为演示使用，所以对真实的url地址进行了打码处理，具体命令：`curl http://xxx/xxx/xxx/xx/xxx`。

然后，我们可以看到返回的状态码是200，证明这个url地址是通的。

curl命令示例2:

```
[app@hr-pro-worker-10 ~]$ curl http://api.beisenappaa.com/Resume/v2/101440/applicant/summary/resume
<!DOCTYPE HTML PUBLIC "-//W3C//DTD HTML 4.01//EN" http://www.w3.org/TR/html4/strict.dtd>
<HTML><HEAD><TITLE>Not Found</TITLE>
<META HTTP-EQUIV="Content-Type" Content="text/html; charset=us-ascii"></HEAD>
<BODY><h2>Not Found</h2>
<hr><p>HTTP Error 404. The requested resource is not found.</p>
</BODY></HTML>
[app@hr-pro-worker-10 ~]$ curl http://api.beisenappaa.com/Resume/v2/101440/applicant/summary/resume
curl: (6) Could not resolve host: api.beisenappaa.com; Unknown error
[app@hr-pro-worker-10 ~]$ curl http://api.aaaaa.com/Resume/v2/101440/applicant/summary/resume
^C
[app@hr-pro-worker-10 ~]$ curl https://www.baidu.com/
^C
[app@hr-pro-worker-10 ~]$ ^C
[app@hr-pro-worker-10 ~]$ curl https://www.baidu.com/ping
```

20 curl命令示例不通情况

通过本示例可以看到不通的情况下，可能会有各种不一样的报错情况。比如404，比如百度这个地址一直卡着没有任何响应，再比如api.aaaaa.com报错Unknown error等等

2.7.2 02.7.2 curl命令语法

curl命令还是一个比较实用和复杂的命令的，但是我们不需要了解那么多的内容，刚开始只需要了解最简单的使用方法即可。

```
curl {选项} {参数}
# 一个利用URL规则在命令行下工作的文件传输工具。
# 我们可以利用这个工具来测试网络是否通，也就是我们利用此工具来检测对方的接口在我们应用服务器上是否能够成功访问，如果是，那么证明网络是通的；如果否，则说明网络不通。
```

2.7.3 02.8 top命令

2.7.4 02.8.1 top命令使用

top命令主要用来查看服务器状态信息的。比如说内存使用情况、CPU使用情况，以及服务器负载情况。

```
top - 09:17:07 up 30 days, 17:23, 1 user, load average 0.72, 0.81, 0.84 3
Tasks: 405 total, 2 running, 403 sleeping, 0 stopped, 0 zombie
%Cpu(s): 20.2 us, 1.1 sy, 0.0 ni, 77.6 id, 0.1 wa, 0.0 hi, 1.0 si, 0.0 st
MiB Mem: 31900.2 total, 1521.4 free, 20475.3 used, 9903.5 buff/cache
MiB Swap: 0.0 total, 0.0 free, 0.0 used. 11084.7 avail Mem
```

	PID	USER	PR	NI	VIRT	RES	SHR	S	%CPU	%MEM	TIME+	COMMAND
1	1767491	root	20	0	6299224	2.5g	57140	S	46.8	8.1	417:17.05	python
2	84678	root	20	0	1096820	605500	42520	R	45.8	1.9	345:39.49	python
3	2813724	root	20	0	6585180	2.8g	57220	S	13.3	9.0	417:12.90	python
4	2816584	root	20	0	6619172	2.9g	57080	S	13.0	9.2	412:15.62	python
5	3908606	root	20	0	1241576	620372	11236	S	9.6	1.9	0:01.75	python
6	2748788	root	20	0	4752524	310340	12104	S	9.3	1.0	2408:16	hostguard
7	3796119	root	20	0	1676892	309504	120592	S	8.3	0.9	4:29.68	soffice.bin
8	34412	root	20	0	316720	69008	17648	S	6.6	0.2	129:34.95	node
9	34929	root	20	0	324096	77156	18400	S	4.7	0.2	129:37.46	node
10	7008	root	20	0	5395924	84208	23196	S	3.0	0.3	1900:38	kubelet
11	5181	root	20	0	4445048	85028	33300	S	2.7	0.3	722:46.99	dockerd
12	6271	root	20	0	1177724	304160	26192	S	2.0	0.9	1292:47	kube-apiserver
13	4647	root	20	0	9634048	108600	10044	S	0.7	0.3	440:59.93	etcd

21 top命令示例1

上述示例1，是在命令行输入top命令之后，回车，就会有如上所示的结果。关于这个结果，我们接下来详细阐述一下我们需要关注的焦点：

1. 在示例1的第一行，最后两个英文单词为：load average，表示为**机器负载**，后面标注的1、2、3，三个数值分别为 1分钟、5分钟、15分钟前到现在的平均值。如果机器是8个CPU，那么如果这三个值，任意一个超过8，那么证明机器负载很高，说明机器压力很大。如果第一个值也就是1分钟的平均值为8以上，那么说明当前服务器压力很大需要找部署老师协助帮忙看看。如果机器是4个CPU，那么这三个值，最好不要超过4，也就是说有几个CPU，平均负载就尽量不要超过几。判断比较简单。
2. %Cpu(s): 20.2 us，这个值代表的是所有CPU的整体使用情况。比如说现在有8个CPU，也就是说目前值是20.2 us，那么意思就是8个CPU的平均值为20.2。这个是一个平均值，如果想查看单个cpu的使用情况，可以在当前界面上输入数字1，然后就会展开单个CPU的使用情况（这个下面会有示例进行讲解）
3. MiB Mem: 表示的是内存的使用情况，在此我们不做展开讲，因为下面有命令free可以查看机器的内存使用情况。


```

35 top - 09:34:30 up 30 days, 17:40, 1 user, load average: 1.12, 0.86, 0.84
36 Tasks: 401 total, 1 running, 400 sleeping, 0 stopped, 0 zombie
37 %Cpu0: 19.7 us, 2.3 sy, 0.0 ni, 75.7 id, 0.0 wa, 0.0 hi, 2.3 si, 0.0 st
38 %Cpu1: 16.7 us, 2.3 sy, 0.0 ni, 79.7 id, 0.0 wa, 0.0 hi, 1.3 si, 0.0 st
39 %Cpu2: 11.5 us, 5.6 sy, 0.0 ni, 81.9 id, 0.3 wa, 0.0 hi, 0.7 si, 0.0 st
40 %Cpu3: 6.3 us, 2.0 sy, 0.0 ni, 91.1 id, 0.0 wa, 0.0 hi, 0.7 si, 0.0 st
41 %Cpu4: 16.4 us, 1.7 sy, 0.0 ni, 80.9 id, 0.0 wa, 0.0 hi, 1.0 si, 0.0 st
42 %Cpu5: 14.0 us, 2.1 sy, 0.0 ni, 83.9 id, 0.0 wa, 0.0 hi, 0.0 si, 0.0 st
43 %Cpu6: 16.4 us, 2.7 sy, 0.0 ni, 80.2 id, 0.0 wa, 0.0 hi, 0.7 si, 0.0 st
44 %Cpu7: 7.1 us, 0.7 sy, 0.0 ni, 90.9 id, 0.0 wa, 0.0 hi, 1.3 si, 0.0 st
45 MiB Mem : 31900.2 total, 2601.8 free, 20540.4 used, 8757.9 buff/cache
46 MiB Swap: 0.0 total, 0.0 free, 0.0 used, 11016.3 avail Mem
47
48
49 PID USER PR NI VIRT RES SHR S %CPU %MEM TIME+ COMMAND
50 2816584 root 20 0 6644716 2.9g 55792 S 43.2 9.4 414:58.23 python
51 3930837 root 20 0 1233592 612564 11072 S 32.6 1.9 0:01.23 python
52 1767491 root 20 0 6309828 2.5g 55820 S 12.6 8.1 419:36.83 python
53 2748788 root 20 0 4752524 310072 12104 S 11.0 0.9 2409:57 hostguard
54 2813724 root 20 0 6584192 2.8g 55864 S 9.3 9.0 419:28.90 python
55 5181 root 20 0 4445048 81344 29616 S 4.0 0.2 723:08.44 dockerd
56 7008 root 20 0 5395924 85384 23376 S 3.7 0.3 1901:25 kubelet
57 6271 root 20 0 1177724 303104 25136 S 2.7 0.9 1293:17 kube-apiserver
58 2816544 root 20 0 113380 7772 6860 S 1.7 0.0 3:08.25 containerd-shim
59 4647 root 20 0 9634048 106760 10280 S 1.0 0.3 441:10.07 etcd

```

22 top命令示例2

上述示例，再我们输入了top命令之后，然后按键数字1，就会把CPU展开显示，如上图所示：

1. 如上图所示，从%Cpu0到%Cpu7，我们可以看到总共有八个CPU，第一列标红的那列，表示的是每个CPU的使用百分比，此示例我们看每个CPU使用都是正常的10%左右，不是特别高，如果有CPU使用超过或者大于90%，而且持续时间比较长，我们要看对应的是不是我们应用的进程，如果是我们的进程，那么要看具体啥任务导致的，长时间占用切使用率超过90%，说明是不太正常的业务。
2. 上图中，我标注了红色的三列，分别为：1、2、3。每一列都是代表了不同的含义：第一列是%CPU，就是当前行的进程使用CPU的情况，我们看数字是43.2%，也就说第一行的进程使用cpu是这么多，这算是正常合理范围的使用情况；第二列%Mem，就是当前行进程使用的内存的百分比（这个百分比是指当前机器总内存的百分比情况），目前我们看是9.4%，如果当前机器内存总量是32G，那么我们可以计算出该进程使用内存大约是3G内存，这个也是一个合理的范围，因为我们进程一般设置内存不会超过4G；第三列，抬头是COMMAND，意思就是进程的名字，我们后端进程都是python进程，前端进程是node进程，文件服务的进程是office.bin，如果有除了这三个进程之外的进程占用量比较高的情况，那么我们需要看是不是客户的服务器的进程或者其他异常进程挤占服务器资源。

我们在使用top命令的时候，只需要关注我们需要关注的点即可，通过该命令我们可以查看客户服务器是否有异常，如果有，那么这种异常可能会导致服务器响应慢等一些问题；如果没有，服务器慢等异常，可能是其他原因导致的。

2.7.5 02.8.2 top命令语法

top命令主要作用就是来查看服务器的使用情况，具体我们需要关注的点在使用里面已经做了详细阐述，其他还有很多其他的功能，我们就不做详述了，那么不便于入门同事的学习。

1	# 命令使用很简单，在命令行上输入top即可，如果想展开查看每个CPU的使用情况，只需要按键数字1即可
2	top

2.8 02.9 df命令

2.8.1 02.9.1 df命令使用

df命令: disk free, 意思就是用于显示目前在Linux系统撒谎给你的文件系统磁盘使用情况统计。其实, 只需要看英文解释即可, 简单直白来讲就是查看磁盘的使用情况。

```

70 root@192:~# df -h
71 Filesystem      1 Size  Used Avail Use% Mounted on
72 tmpfs           3.2G  5.6M  3.2G   1% /run
73 /dev/vda1       2 99G   46G   49G  49% /
74 tmpfs           16G    0    16G   0% /dev/shm
75 tmpfs           5.0M    0   5.0M   0% /run/lock
76 192.168.1.157 255G  154G   89G  64% /data
77 tmpfs           3.2G   76K  3.2G   1% /run/user/132
78 tmpfs           16G   12K   16G   1% /var/lib/kubelet/pods/bc04d630-bb6b-424a-aa3f-1a04f668058b/volu
-   ected/kube-api-access-7vk4m
79 overlay         99G   46G   49G  49% /var/lib/docker/overlay2/188361f5f4d1aba02fc3379a12a0017c3020c3
-   4ec4/merged
80 shm            64M    0   64M   0% /var/lib/docker/containers/5c6f2691e262f6f12375fff967f8c85e1a68
-   113700/mounts/shm
81 overlay         99G   46G   49G  49% /var/lib/docker/overlay2/fa7c017caea3cea61c09ad46b956e18ca82b9a
-   80cf/merged
82 tmpfs           16G   12K   16G   1% /var/lib/kubelet/pods/8d2c289d-e70f-4a92-b761-d5a5bcdcd9ba/volu
-   ected/kube-api-access-blfd
83 overlay         99G   46G   49G  49% /var/lib/docker/overlay2/5ad8b6cbc432f3975469e434a7ec9e6722a722
-   f997/merged
84 shm            64M    0   64M   0% /var/lib/docker/containers/d29bb8a8b66640dc78819308abac5eba001d
-   72b75e/mounts/shm
  
```

23 df命令示例1

上述示例1中, 我们可以看到使用的命令是df -h命令, 关于这个命令的输出结果, 我们做一个详解:

1. 红色标注的第一行, 有几个标题, 我们可以分别看一下filesystem Size used Avail Use% Mounted on。我们主要关注的是我用红色框框标记的部分: 第一个Size, Size表示的是磁盘的总大小, 下面两行需要我们关注的部分的大小分别为: 99G、255G。第二个Used, 表示目前已经使用的磁盘大小是多少, 比如99G哪一行的Used为46G; 第三个Avail, 表示还有多少可用磁盘, 即还有多少空余没有使用的磁盘; 第四个Use%, 表示当前磁盘的使用率是多少, 我们99G这一行, Use%为49%, 说明磁盘已经使用了49%, 还有51%我们可以使用。
2. 正常情况下, 应用服务器会有如上图所示的/var/lib/docker等等这种情况, 这个情况可以不用关注。我们只需要关注两个情况, 一般是我们需要关注Mounted on的挂载点为根 (/) 的情况, 也就是第二行标红的部分, 这个挂载使用率不能超过80%, 还有一个就是数据盘, 一般为/data等或者类似的目录, 就如我们第三处标红的部分, 这个我们使用率要求同样不要超过80%, 如果超过了, 系统可能会有问题, 我们建议要及时清理磁盘或者对磁盘进行扩容操作, 具体操作可以咨询部署老师。

2.8.2 02.9.2 df命令语法

df命令是一个比较简单的命令

我们来看一下df命令的详解: report file system disk space uage。通过这个详解我们可以一目了然就知道, 该命令是用来查看磁盘的使用情况的。

我在df命令后面加了一个参数-h, 来看一下这个参数的意思

```

1 df -h
2 # -h, --human-readable
  
```


3 # -h 参数的意思，就是显示内容为人类可读，就是会让人类读者舒服的方式进行显示，要不然不加 -h，会以字节的方式显示，我们就没法一目了然的了解具体的机器使用情况了

2.9 02.10 free命令

2.9.1 02.10.1 free命令使用

free命令，也是一个比较简单的命令，主要作用就是查看当前机器的内存的使用情况。

```
362 root@192:~# free -h
363      total        used        free      shared  buff/cache   available
364 Mem:       31Gi       19Gi       2.7Gi       28Mi       8.7Gi       11Gi
365 Swap:         0B         0B         0B
366 root@192:~#
```

24 free命令示例1

free命令示例如图所示：

1. 我们先看一下标题栏：total（表示该机器的总内存情况） used（当前服务器使用的内存多少） free（空闲内存有多少） buff/cache（缓存使用的内存情况），我们只需要关注这四列即可
2. 其实，我们看一下used内存和free内存是不是合理范围即可，used内存只要不超过机器的80%即可，比如我们现在总内存是31G，那么80%即24.8G，只要不超过这个内存，说明服务器内存还是正常合理范围内，不是很紧张。

2.9.2 02.10.2 free命令语法

free命令：Display amount of free and used memory in the system。简单来讲，就是查看系统内存的使用情况。

参数-h的意思详见示例，主要也是方便人类可读。

```
1 free -h
2 # -h, --human
3 #          Show all output fields automatically scaled to shortest
4 #          three digit unit and display the units of print out. Follow-
#          ing units are used.
```

2.10 02.11 date命令

date命令，更是一个非常简单非常简单的命令，输入之后，直接回车即可。

```
368 root@192:~# man free
369 root@192:~# date
370 Sun Apr 7 11:03:17 AM CST 2024
371 root@192:~#
```

25 date命令查看服务器时间示例1

date命令的作用就是查看当前服务器的时间

3 03 K8s相关的操作命令

注意k8s的命令和linux的基础命令一样都是命令与后面的参数之间需要空格隔开的!!!

3.1 03.1 kubectl命令

kubectl命令是k8s集群的管理命令。这是一个基础命令，其他所有的命令都是在这个命令的基础上进行使用 and 操作的。

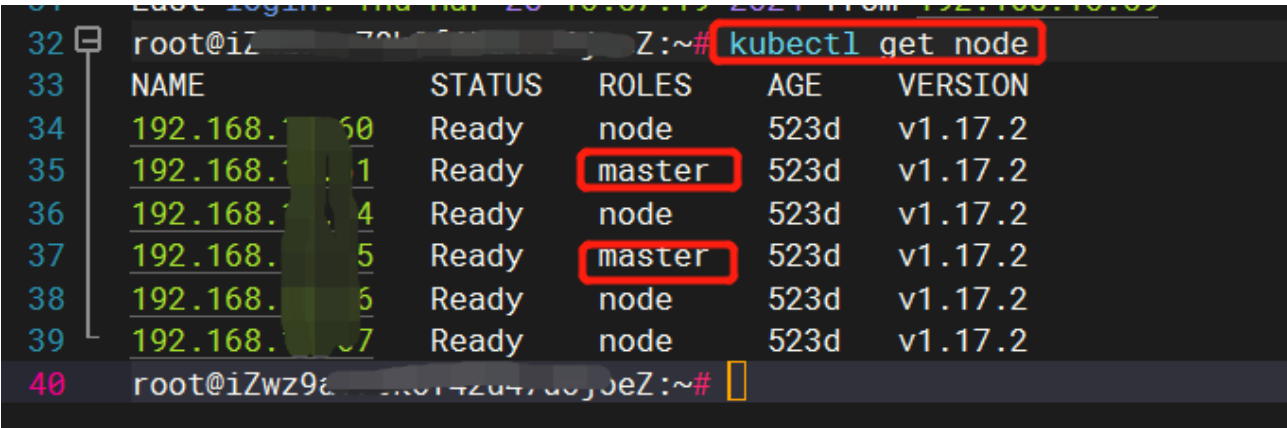
```

1 kubectl help
2 # 如果不知道命令如何使用，可以通过help进行查看命令的详细说明
    
```

3.2 03.2 kubectl 子命令get

kubectl的子命令get用途：查看节点信息，查看pod信息等

3.2.1 03.2.1 get子命令查看节点信息



26 kubectl查看节点信息示例1

```

1 # 命令格式如下，get node即可以获取k8s集群的node信息
2 kubectl get node
3 # 下面为输出结果
4 NAME            STATUS    ROLES    AGE    VERSION
5 192.168.0.60     Ready    node     523d   v1.17.2
6 192.168.0.61     Ready    master   523d   v1.17.2
7 192.168.0.64     Ready    node     523d   v1.17.2
8 192.168.0.65     Ready    master   523d   v1.17.2
9 192.168.0.66     Ready    node     523d   v1.17.2
10 192.168.0.67     Ready    node     523d   v1.17.2
11
    
```

```

12 # 通过输出结果，我们可以看到该集群有俩个master和四个node节点（node节点即为worker节
13 # 点）。一般情况下，我们master节点是不作为工作节点使用的，master只是整个集群的入口，但是
14 # 测试环境和一些特殊的环境除外（比如说客户正式环境资源有限的情况下除外）
15 # 比如下面这种情况
16 kubectl get node
17 NAME                STATUS    ROLES    AGE    VERSION
18 192.168.0.59        Ready    master   554d   v1.17.2
19 # 由于只有一台服务器，所有该服务器既是worker节点也是master节点。
20 # VERSION v1.17.2是k8s的版本号
21 # STATUS 是k8s节点的运行状态，如果显示为NotReady则表示不可用，需要进行修复，请联系
    部署老师。

```

此处，我们看你需要多讲一点，方便后续多余整个k8s相关命令的理解，以及后续离线环境更新等。

一般项目测试环境是只有一台服务器的，所以对于离线环境更新来说只需要处理一台服务器即可。对于一些大项目，测试环境可能不止一台，那么更新环境的时候需要处理的可能不止一台服务器（如果不是特别清楚的情况，可以咨询部署老师）。

对于该命令，大家只需要了解该命令的用途即可。

3.2.2 03.2.2 get子命令查看pod信息

```

1 kubectl get pod
2 # 以下为执行kubectl get pod命令的输出结果
3 NAME                                READY    STATUS    RESTARTS    AGE
4 celery-beat-5fb59494c6-mxn2f        1/1      Running   0            12d
5 celery-quick-dd7bdc486-cn5pf        1/1      Running   0            12d
6 celery-slow-67986d56b9-89frc        1/1      Running   0            12d
7 default-http-backend-7c9558dbb6-pdf 1/1      Running   7            415d
8 deploy-server-65d78f8d8b-qb82v      1/1      Running   8            415d
9 deploy-server-schedule-7dc9fc44dd-kk 1/1      Running   9            415d
10 elasticsearch-7d7974d7cd-w9frq      1/1      Running   0            157d
11 hcm-cloud-6b96f8f889-6bc79          1/1      Running   0            12d
12 hcm-cloud-6b96f8f889-f9smh          1/1      Running   0            12d
13 hcm-core-7f77d86578-67czb          1/1      Running   0            12d
14 hcm-office-5c549c7c7c-gs2zv         1/1      Running   108          554d
15 hcm-script-7c797b9dc7-pjx7h         1/1      Running   30           12d
16 nginx-ingress-controller-bkkqs       1/1      Running   9            554d
17 redis-656865c997-dnxtj              1/1      Running   7            547d
18 redis-background-6b68b554b6-ml5qt    1/1      Running   0            148d
19 redis-monitor-68d449c998-l4l6g       1/1      Running   7            554d
20 # 根据输出结果，我们需要关注的内容如下

```

```

21 # READY状态是不是 1/1, 比如0/1 (同时, status 为Running状态) 说明服务器启动还没有成
    # 功, 需要继续等待
22 # 如果STATUS非Running状态等其他状态, 说明服务不可用。请联系部署老师帮忙分析查看
23 # AGE列为pod的运行时间长度, 后面的单位d为day的缩写, 也就是天数; 刚开始启动运行单位是s
    # 代表是秒, 然后是m代表是分钟, 然后是h代表是小时
24
25 kubectl get pod -owide
26 # 该命令可以查询更详细的信息, 比如一个pod是运行在那台服务器上等信息
27 NAME                                READY   STATUS    RESTARTS   AGE
   IP                                NODE    NOMINATED NODE   READINESS GATES
28 celery-beat-5fb59494c6-mxn2f        1/1     Running   0           12d
   172.20.0.234 192.168.0.59 <none>         <none>
29 celery-quick-dd7bdc486-cn5pf        1/1     Running   0           12d
   172.20.0.238 192.168.0.59 <none>         <none>
30 celery-slow-67986d56b9-89frc        1/1     Running   0           12d
   172.20.0.240 192.168.0.59 <none>         <none>
31 default-http-backend-7c9558dbb6-pdffv 1/1     Running   7
   415d 172.20.0.59 192.168.0.59 <none>         <none>
32 deploy-server-65d78f8d8b-qb82v      1/1     Running   8
   415d 172.20.0.71 192.168.0.59 <none>         <none>
33 deploy-server-schedule-7dc9fc44dd-kk7l9 1/1     Running   9
   415d 172.20.0.67 192.168.0.59 <none>         <none>
34 elasticsearch-7d7974d7cd-w9frq      1/1     Running   0
   157d 172.20.0.111 192.168.0.59 <none>         <none>
35 hcm-cloud-6b96f8f889-6bc79          1/1     Running   0           12d
   172.20.0.231 192.168.0.59 <none>         <none>
36 hcm-cloud-6b96f8f889-f9smh          1/1     Running   0           12d
   172.20.0.237 192.168.0.59 <none>         <none>
37 hcm-core-7f77d86578-67czb          1/1     Running   0           12d
   172.20.0.241 192.168.0.59 <none>         <none>
38 hcm-office-5c549c7c7c-gs2zv         1/1     Running   108
   554d 172.20.0.50 192.168.0.59 <none>         <none>
39 hcm-script-7c797b9dc7-pjx7h        1/1     Running   30          12d
   172.20.0.233 192.168.0.59 <none>         <none>
40 nginx-ingress-controller-bkkqs      1/1     Running   9
   554d 172.20.0.68 192.168.0.59 <none>         <none>
41 redis-656865c997-dnxtj             1/1     Running   7
   547d 172.20.0.51 192.168.0.59 <none>         <none>
42 redis-background-6b68b554b6-ml5qt   1/1     Running   0
   148d 172.20.0.122 192.168.0.59 <none>         <none>
43 redis-monitor-68d449c998-l4l6g      1/1     Running   7
   554d 172.20.0.66 192.168.0.59 <none>         <none>

```

3.3 03.3 kubectl子命令logs

```

1 # 假设, 我们要查看pod: hcm-core-7f77d86578-67czb 的日志, 那么命令如下
2 kubectl logs -f --tail=200 hcm-core-7f77d86578-67czb
3 # -f 参数是然日志进行流式输出, 默认我们需要加上这个参数

```

4 # --tail=200 查看该pod的最后两百行进行输出，同时会一直刷新最新接收到的日志到当前屏幕

```

249 root@iZwz9f05rbgm7bbczwuceZ:~# kubectl logs -f --tail=200 hcm-core-7f77d86578-67czb
250 <stdin>:98:1: 'core.ds.formula.engine.BaseFormulaObject as BaseSalaryFormulaObject' imported but unused
251 <stdin>:99:1: 'core.ds.formula.engine.BaseCustomerFormulaObject' imported but unused
252 <stdin>:100:1: 'apps.workflow.workflow_util.WorkFlowDynamicScriptBase' imported but unused
253 <stdin>:100:1: 'apps.workflow.workflow_util.WorkFlowDynamicScriptExecutorBase' imported but unused
254 <stdin>:101:1: 'core.ds.formula.engine.BaseFormulaObject as BaseSocialFormulaObject' imported but unused
255 <stdin>:102:1: 'core.base.utils.mobile.MobileMessageServiceBase as HCMMobileMessageService' imported but unused
256 <stdin>:103:1: 'uuid' imported but unused
257 <stdin>:104:1: 'pika' imported but unused
258 <stdin>:105:1: 'util' imported but unused
259 <stdin>:106:1: 'functools.reduce' imported but unused
260 <stdin>:107:1: 'tornado.escape.to_unicode' imported but unused
261 <stdin>:108:1: 'decimal' imported but unused
262 <stdin>:109:1: 'core.manage.auth.handler_exception.LoginRedirectException' imported but unused
263 <stdin>:110:1: 'ftplib.FTP' imported but unused
264 <stdin>:111:1: 'apps.time.formula.formula.BaseTimeFormulaObject' imported but unused
265 <stdin>:112:1: 'core.manage.auth.saml_util.SAMLRequestHandler' imported but unused
266 <stdin>:113:1: 'core.extend.third.base.sso_util.SsoUtil' imported but unused
267 <stdin>:114:1: 'jwt' imported but unused
268 INFO 140298994566912 [handlers.py:240] end openapi [hcm.formula.check] in 31 ms with

```

27 kubectl logs -f --tail=200命令示例1

依照上述示例，我们当我们需要在真实环境中查看指定pod日志的时候，我们只需要对pod名字进行替换即可。当然，我们不知道具体pod名字的时候，我们需要通过命令kubectl get pod查看具体pod名字。

3.4 03.4 kubectl子命令exec（命令作用：进入容器内部）

为啥需要进入容器内部呢？

1. 需要查看容器内部网络通不通，具体测试网络通不通的方法和linux服务器上测试方式一致，只是进入容器这个步骤是比在linux多的一个步骤
2. 在容器内部登录数据库服务器
3. 执行表结构调整脚本或者index差异调整脚本

```

1 # 一定要注意：每个命令直接都是有空格的，或者说参数与命令直接都是有空格的！
2 kubectl exec -it {pod_name} bash
3 # 举例子
4 root@192:~# kubectl exec -it hcm-core-bfb686c77-j2c7q bash
5 kubectl exec [POD] [COMMAND] is DEPRECATED and will be removed in a future
  version. Use kubectl exec [POD] -- [COMMAND] instead.
6 root@hcm-core-bfb686c77-j2c7q:/hcm_server#
7 # 一定要注意：进入pod的标志是root@后面的名字由主机名变成了pod名字。
8 # 进入pod我们使用完成之后，如何退出pod呢？ 在当前命令行上输入exit命令即可
9 root@hcm-core-bfb686c77-j2c7q:/hcm_server# exit
10 exit
11 root@192:~# （到这里表示已经退出了pod，主机名由pod_name变成了原来的主机名192）

```

3.5 03.5 kubectl子命令scale

```

1 # scale子命令的作用是 调整pod个数，有需要对pod个数进行调整的时候会用到这个子命令
2 kubectl scale deployment hcm-core --replicas=5
3 # kubectl是k8s操作的命令，scale是kubectl的子命令，deployment是k8s的pod的一种类型
  （我们一般调整的就是这种类型，所以默认就是deployment）

```

```
4 # --replicas=5 表示将pod个数调整为5个，这里数值设置为数字几，就是调整为几个的意思。
5 # replicas的英译为：副本、复制品的意思
6 # hcm-core是我们后端pod的名字，我们所有服务的名字列表如下：
7 hcm-core          后端服务的名字
8 hcm-client        前端服务的名字
9 hcm-office        文件服务的名字
10 hcm-script        脚本服务的名字
11 redis            redis服务的名字
12 redis-background redis-background服务的名字
13 redis-monitor    redis-monitor服务的名字
14 celery-beat      队列任务的名字
15 celery-quick     消息发送队列的名字
16 celery-slow      后台任务队列的名字
17 celery-XXXX      队列可能不止上面这三种，如果还有其他的那么以此类推
```

3.6 03.6 kubectl子命令describe（命令选学）

```
1 # 注意{pod_name}是一个占位符，在实际使用的过程中，需要用实际值替换这个占位符
2 kubectl describe pod {pod_name}
3 # 比如一个pod的状态不是Running状态，我们可能需要查看pod为啥没有启动成功的情况，这个时候，
4 # 我们需要describe查看一下pod启动不成功的原因
5 # 这也是describe这个子命令的作用
6 root@192:~# kubectl describe pod hcm-core-bfb686c77-j2c7q
7 Name:          hcm-core-bfb686c77-j2c7q
8 Namespace:     default
9 Priority:       0
10 ...
11 ...
```


4 04 数据库命令（以mysql为例，如果客户买的是其他数据库，则有对应的厂商进行对应的运维操作）

注意：数据库操作一般情况是禁止单人单独进行操作的！！！所有操作必须有其他同事或者懂数据库的客户在场的情况下进行操作，或者让客户自己操作，一般人员禁止随意或单独操作数据库。如有操作数据库需求请联系项目经理或者区总等负责人，确认必须要操作，然后联系部署老师协助帮忙查看操作是否有问题！

数据库操作必须先备份、先备份、先备份（重要的事情，强调三遍），再操作！！

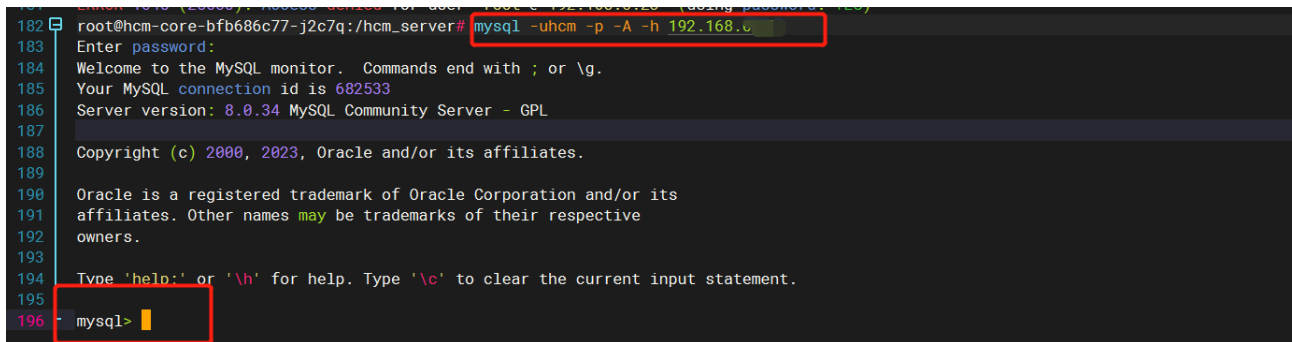
一般数据库为mysql的项目，一般的数据库操作由我们自己人来处理。非mysql的情况，一般客户是购买的厂商的数据库，这个情况下，可能需要联系厂商人员或者客户的数据库运维人员对数据库进行操作。

以下所有的操作皆以mysql为例，因为我们也只对mysql进行简单的运维操作，其他数据库问题需要联系对应数据库厂商进行。

但是要求现场的同事，无论客户的数据库是可以通过UI登录的还是通过命令行进行登录的，需要现场同事知道如何登录数据库，可能每个数据库的登录方式是不一样的。

4.1 04.1 登录数据库操作

- 1 # 注意命令行中的空格情况，所有花括号内部的中文需要使用实际数值进行替换操作，同时需要去掉花括号
- 2 mysql -u{用户名} -p{密码} -P {端口号} -A -h {数据库IP}
- 3 mysql -uroot -p -A -h 192.168.0.1
- 4 # 具体实例请见下图



```
182 root@hcm-core-bfb686c77-j2c7q:/hcm_server# mysql -uhcm -p -A -h 192.168.0.1
183 Enter password:
184 Welcome to the MySQL monitor.  Commands end with ; or \g.
185 Your MySQL connection id is 682533
186 Server version: 8.0.34 MySQL Community Server - GPL
187
188 Copyright (c) 2000, 2023, Oracle and/or its affiliates.
189
190 Oracle is a registered trademark of Oracle Corporation and/or its
191 affiliates. Other names may be trademarks of their respective
192 owners.
193
194 Type 'help;' or '\h' for help. Type '\c' to clear the current input statement.
195
196 mysql>
```

28 mysql登录操作示例1

上述示例是以命令行mysql登录操作为例：一般情况下，禁止在命令行中输入密码（-p参数后面不要输入密码，回车之后，会有一个Enter password的提示，在这个提示后面输入密码，密码输入的过程中不会显示密码，确保输入正确之后，回车即可）

1. 上图中的第一个标红的部分为登录数据库的命令
2. 标红的第二部分 `mysql>` 该标志，意味着我们已经登录到数据库里面了，既可以在数据库中进行相应的操作了
3. 使用完数据库之后，记得退出数据库，退出的命令为：`\q`（输入`\q`然后回车即可）

4.2 04.1.1 数据库服务器上登录

在数据库服务器上登录的时候我们可以不需要添加后面的-h参数，比如

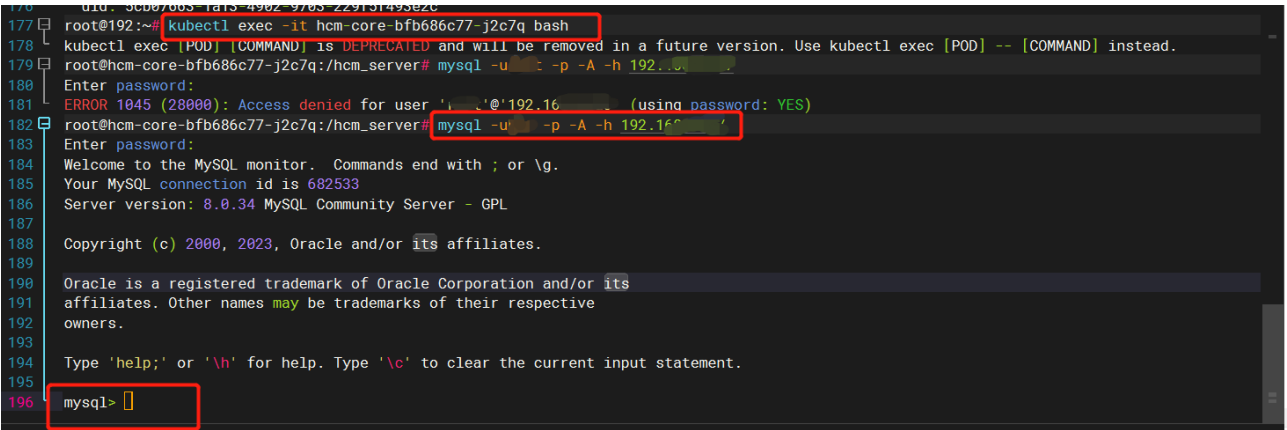
```

1 # 在数据库服务器上，使用下列命令既可以登录数据库操作
2 mysql -u{用户名} -p{密码} -A
    
```

4.3 04.1.2 容器内登录数据库操作

```

1 # 首先执行进入容器的命令
2 # kubectl exec -it {pod_name} bash
3 kubectl exec -it hcm-core-bfb686c77-j2c7q bash
4 # 进入容器之后，执行登录mysql数据库的命令即可，详情见下图所示
    
```



29 容器内执行mysql示例1

上述示例的具体操作步骤：

1. 首先执行kubectl exec -it {pod_name} bash命令进入容器内（一定要记得查看是否正确进入到容器内）
2. 在容器内执行mysql登录命令
3. 在显示mysql>的提示的时候，代表已经登录到数据库内

4.4 04.2 数据库查看慢sql命令

当数据库服务器异常的时候，比如负载异常、io异常等各种不正常情况，或者环境慢，需要查看是不是数据库堵住了导致的慢问题，这个时候需要登录数据库，并查看数据库中sql的执行情况。

```

1 # 注意前面的mysql>只是一个mysql输入命令的一个提示符，表示你已经进入数据库，后面的命令才是我们实际要在数据库中要执行的命令。
2 mysql> select trx_query,trx_mysql_thread_id,trx_state, trx_started from
    information_schema.INNODB_TRX;
    
```

- 3 Empty set (0.00 sec)
- 4 # 上述提示Empty set表示现在数据库中并没有慢sql在执行

```
198
199 mysql> select trx_query, trx_mysql_thread_id, trx_state, trx_started from information_schema.INNODB_TRX;
200 Empty set (0.00 sec)
201
202 mysql>
```

30 查看慢sql命令示例1

上述示例，需要执行的命令。可以直接进行拷贝粘贴防止手敲出现错误。

4.5 04.3 数据库查看完整sql命令

在04.2的示例中由于我们没有看到有慢sql的情况，这个我们没法去通过ID进行查询，我再找一个新的示例进行讲解如何查看慢sql

```
mysql> select trx_query, trx_mysql_thread_id, trx_state, trx_started from information_schema.INNODB_TRX;
+-----+-----+-----+-----+
| trx_query | | trx_mysql_thread_id | trx_state |
+-----+-----+-----+-----+
| NULL | | 6 | RUNNING | 2024-0
4-08 10:40:16 | | | |
| select ` | | | |
ata_object | | | |
4-08 10:35:02 | | | |
+-----+-----+-----+-----+
2 rows in set (0.00 sec)

mysql> select * from information_schema.processlist where id=858387;
Empty set (0.00 sec)

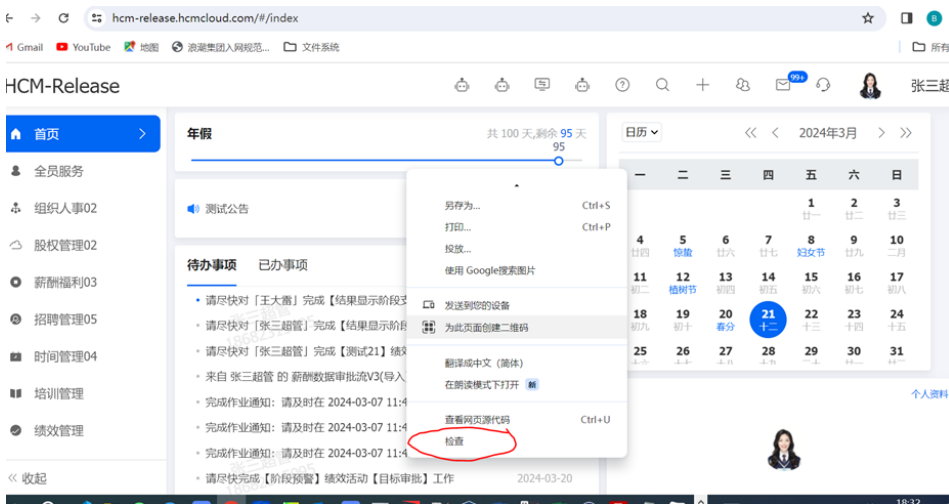
mysql>
```

31 mysql查慢sql示例1

5 05 环境慢问题处理

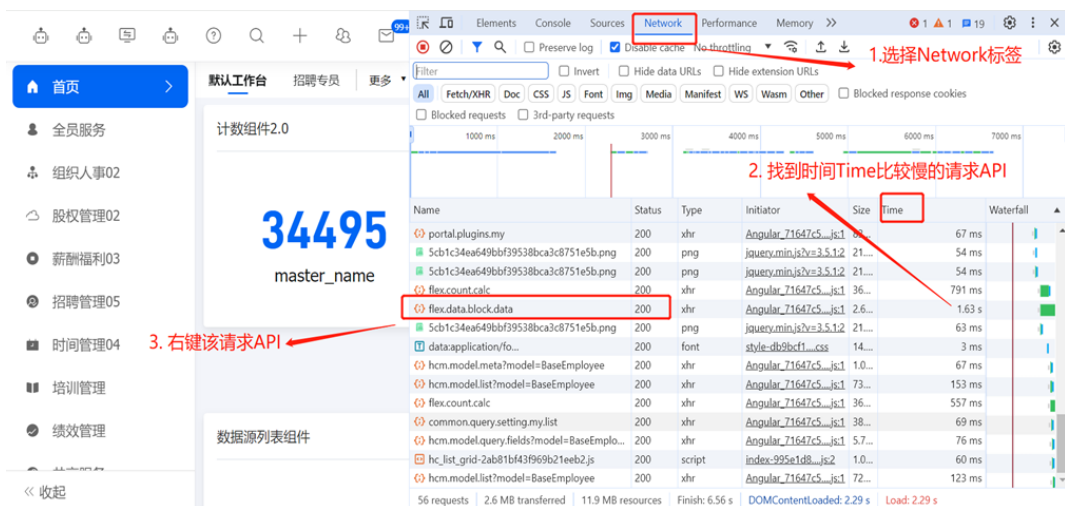
5.1 05.1 如何分析慢api

当我们访问一个页面加载慢或者报错、网络开小差等各种情况，这个时候需要我们去查看一下浏览器的具体的网络请求情况，根据具体情况进行具体分析。



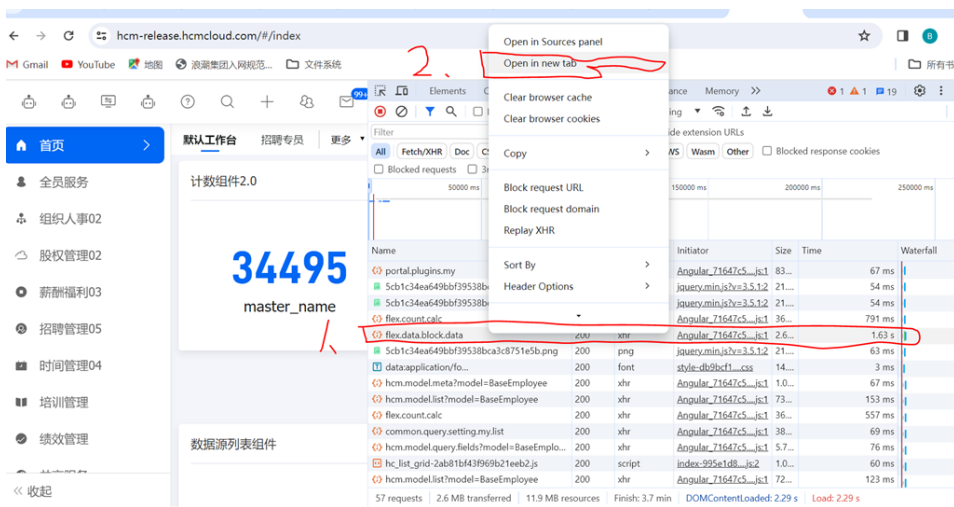
32 打开浏览器检查

~我们需要打开浏览器的检查功能，这个操作，就是在浏览器的当前窗口的空白处，右键就会出现如上图所示的情况，然后点击红色检查部分



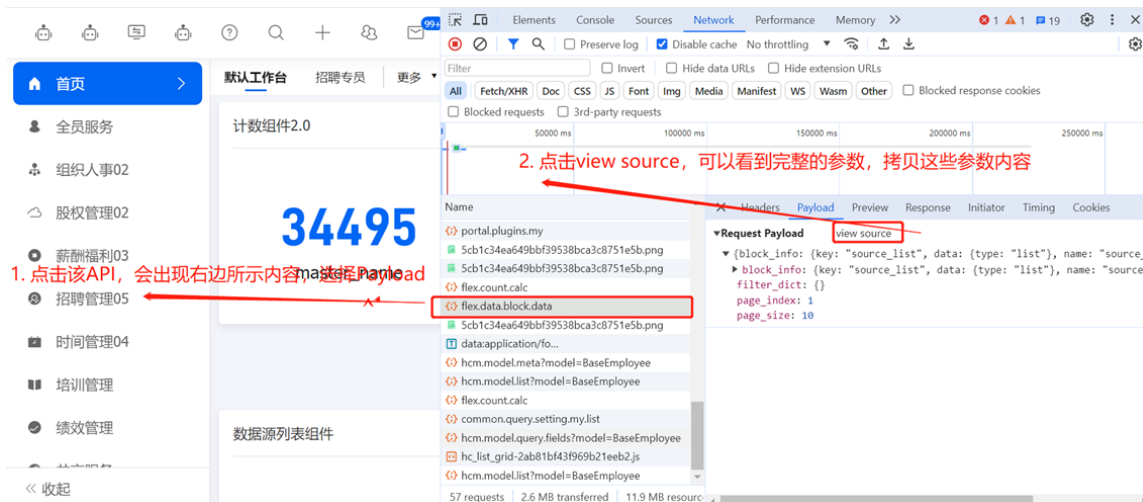
33 打开检查之后查看NetWork或者中文名为网络的部分

2. 点击“检查”之后，我们可以看到如上图所示的情况，然后重新刷新当前页面，查看所有的api请求，哪个api请求耗时比较慢，然后鼠标选中请求比较慢的api，右键点击改API



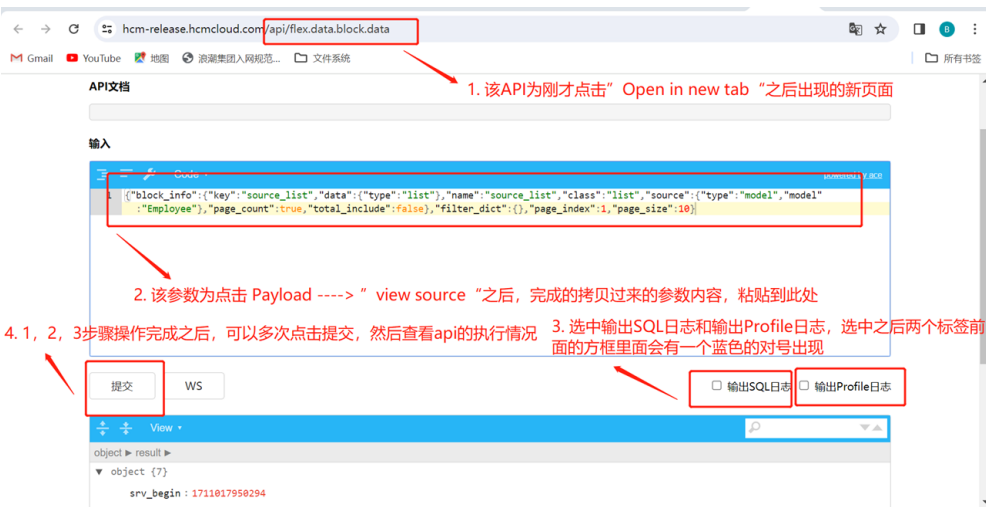
34 查看慢API

3. 右键慢API之后，选择“Open in new tab”，会跳转到一个新的浏览器标签页里面。



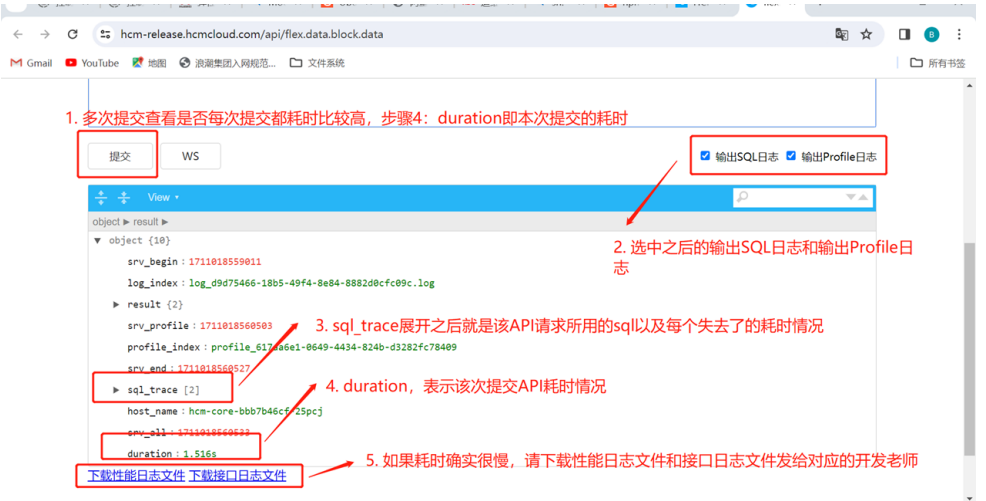
35 拷贝慢API的参数

4. 拷贝慢API的参数，具体操作可以查看图片所示。如果环境加密了，那么传递的参数也会进行加密处理，这个时候拷贝出来的数据需要解密之后才能使用（请注意）



36 手动请求API看是否也一样很慢

5. 手动通过api进行多次请求，查看每次请求的耗时情况，是否每次都是特别慢，如果是，请按照图示进行相关的操作。



37 获取sql和profile日志

6. 获取慢sql和性能日志，并下载出来发给对应的模块的开发老师

到此，基本环境慢的情况，可以根据这个示例进行追踪。如果环境出现报错或网络开小差等情况，需要具体问题具体分析。

5.2 05.2 API报错和网络开小差情况分析

具体API报错情况，可以找二开老师或者找对应的模块的老师去查看kibana日志，如果是测试环境，直接上服务器查看后端（即hcm-core）的日志即可。

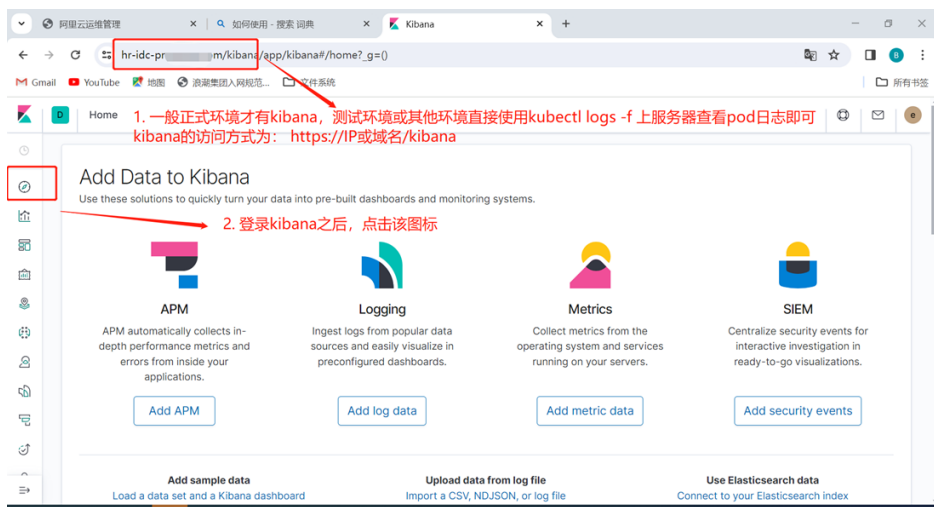
6 06 kibana的简单使用

6.1 06.1 如何登录kibana

一般kibana的访问方式，即https://域名或IP/kibana

具体账号密码，请咨询部署老师。

6.2 06.2 使用kibana进行查询



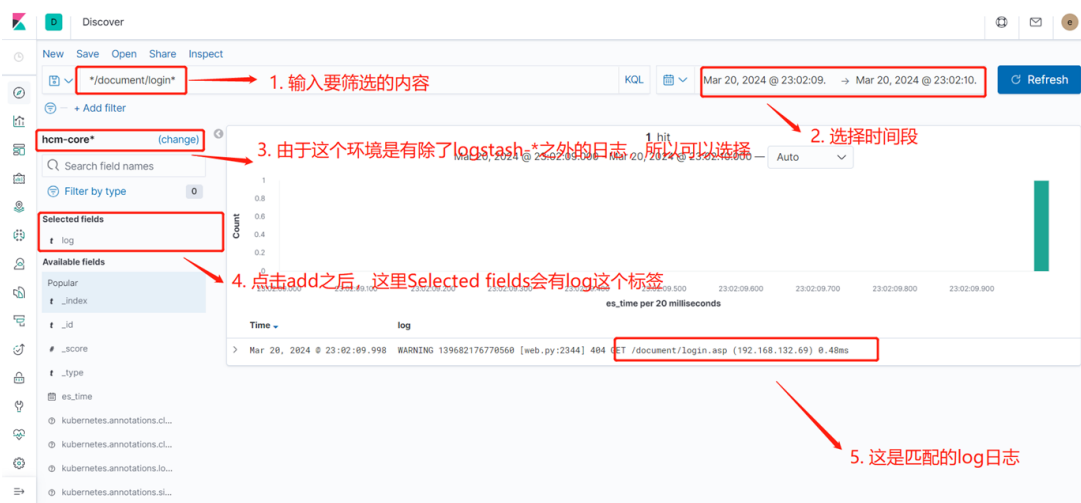
38 kibana使用实例1

登录kibana之后，会看到如上图所示的界面，具体操作请遵循实例中的讲解进行操作。



39 kibana使用示例2

2. 具体详解见上图中所示



40 kibana使用示例3

3. 具体操作见上图所示的详解

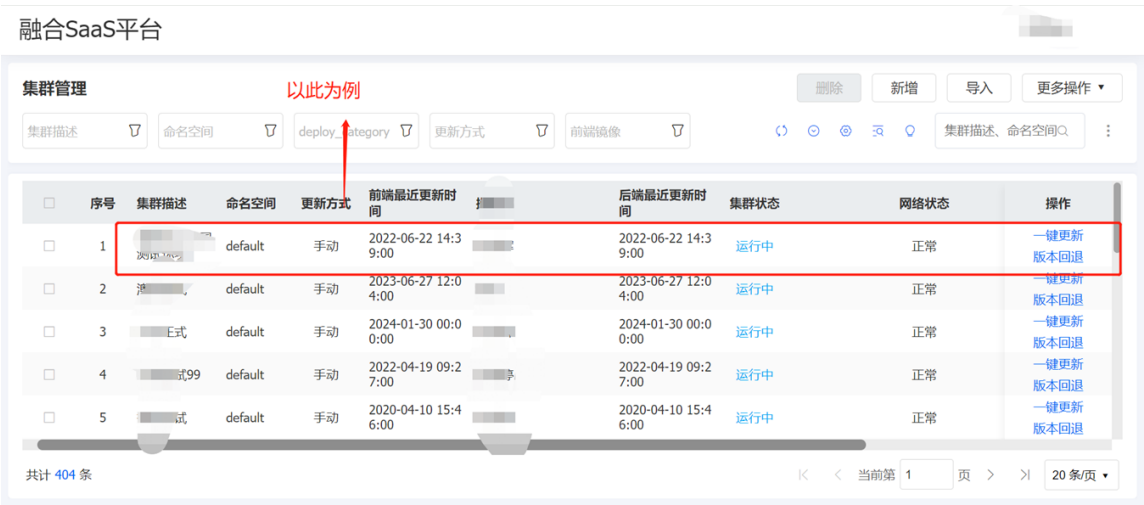
7 07 申请离线镜像包

7.1 07.1 如何知道项目环境的架构等详细信息

每个项目的具体情况，如果不是特别了解，请先咨询部署老师。如果没有ops⁵（这是超链接可以直接点击打开登录）登录账号，请咨询部署老师，将清楚自己所负责的项目等各种可能必要的信息。

7.2 07.2 如何申请离线镜像包

在获取了ops登录权限之后，登录ops，查看自己所管理的项目



41 登录ops之后，会看到自己所管理或负责的项目1

登录ops之后，会看到如上图所示的情况，会有最少一个自己所负责的项目展示出来，如果一个都没有，请联系部署老师，要求给添加自己所负责的项目的权限。



42 ops示例2

5. <https://inspur.hcmcloud.cn/ops#/index>

2. 具体操作请见上图示例中的详细讲解



43 ops示例3

3. 架构需要咨询一下部署老师，然后选择适合自己项目的架构；生产还是**release**也需要根据具体情况进行选择，一般环境都是生产的代码。这两件事情如果不是特别了解请咨询部署老师。

镜像类型一般是两种：前端镜像和后端镜像，前端镜像对应的英文就是**client**，后端镜像对应的英文就是**server**。

当你选择的是前端镜像的时候，后面的数据库类型会自动隐藏掉，因为前端镜像是不管数据库是什么类型的。

当你选择的是后端镜像的时候，后面的数据库类型需要选择对应的数据库，如果数据库具体不清楚请咨询部署老师。

上次版本镜像号，这个版本号需要通过访问当前项目的网址：<https://域名或IP/client/version> 或 <https://域名或IP/server/version>，这是查看上次版本号，所谓的上次版本号也就是当前大家项目环境的版本号是多少。有一些的版本号中除了阿拉伯数字之外可能包含了一下英文字符，这种情况下，请忽略阿拉伯数字之外的所有英文和特殊字符，只需要拷贝阿拉伯数字即可。

本次版本号，如果是开发老师要项目上更新环境，请咨询开发老师要求的最低的版本是多少。如果只是项目上自己需要对环境进行更新，则可以查看我们公有云的当前版本号当作自己所需要升级的版本号即可，比如，前端镜像版本：<https://inspur.hcmcloud.cn/client/version>⁶，后端镜像版本号：<https://inspur.hcmcloud.cn/server/version>⁷

上面的所有操作执行完成之后，点击“确认”按钮。

6. <https://inspur.hcmcloud.cn/client/verion>

7. <https://inspur.hcmcloud.cn/client/verion>



44 离线包升级示例

4. 执行完步骤三之后，选中自己刚新增的那条记录，选中之后，点击“更多操作”。上图示例中，我们可以看到两个选项：建议非必要不要使用“申请全量离线包”，有一些项目由于差异离线包执行会报错，所以只能申请全量离线包。

全量离线包和差异离线包的区别：全量离线包非常大，最少1.5G；而差异离线包可能只有100M左右，这样对于大家环境更新的时候文件上传会非常友好。推荐大家点击的时候使用“差异离线包”操作。

5. 点击完成“申请差异离线包”之后，烦请等待15分钟左右，再次登录ops查看离线文件名是否已经生成，如果已经生成，请到离线服务器上获取对应的名字的离线文件，并下载。

7.3 07.3 上传离线镜像包到服务器更新环境

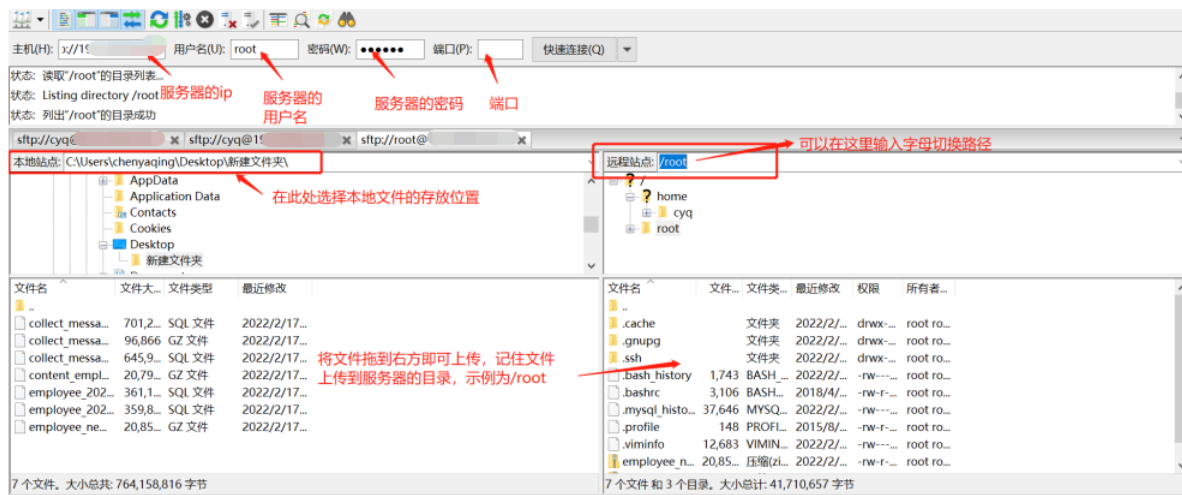
先将ops上构建的离线镜像包下载到本地电脑，然后登录到客户的应用服务器(有多台应用服务器的登录到其中一台即可)

- 1. 获取需要导入镜像包的所有worker节点IP地址

```
kubectl get node --show-labels |grep hcm-node |awk '{print $1}'
```

```
[root@gyjt-hbcs-xcecs-uat-ehr-ecs1-poc--01 ~]# kubectl get node --show-labels |grep hcm-node |awk '{print $1}'
0.1.14.103
0.1.8.14.107
0.1.14.21
0.1.14.80
0.123.14.84
```

- 2. 将ops上构建的离线镜像包上传到其中一台worker服务器的/data/nfs/hcm目录下(只需要传到其中一台worker服务器的/data/nfs/hcm目录下后，其他worker节点都可以共享此文件)



3. 分别登录到每台worker机器上导入镜像包

`docker load -i /data/nfs/hcm/前端镜像包文件名`

`docker load -i /data/nfs/hcm/后端镜像包文件名`

```
[root@16-300g-0001 ~]# docker load -i client-121725.tar.gz
0943da50267e: Loading layer [=====>] 1.776MB/1.776MB
bdaff6736761: Loading layer [=====>] 2.048kB/2.048kB
b19ff59f843d: Loading layer [=====>] 2.048kB/2.048kB
c997bed67804: Loading layer [=====>] 11.32MB/11.32MB
9e0a080da32: Loading layer [=====>] 602.4MB/602.4MB
387f704e98f5: Loading layer [=====>] 106kB/106kB
Loaded image: ccr.ccs.tencentyun.com/hcm/client.release:121725
[root@16-300g-0001 ~]# docker load -i server-121736.tar.gz
c0389ccde622: Loading layer [=====>] 1.281GB/1.281GB
3db55a51ec56: Loading layer [=====>] 12.88MB/12.88MB
7a20aa2bc151: Loading layer [=====>] 213.6MB/213.6MB
f2e3341b6835: Loading layer [=====>] 145.3MB/145.1MB
1d57f9c09f42: Loading layer [=====>] 32.99MB/32.99MB
883932b9f3b4: Loading layer [=====>] 30.65MB/30.65MB
e1665b499f1a: Loading layer [=====>] 8.815MB/8.815MB
e3109b97b1a8: Loading layer [=====>] 68.11MB/68.11MB
0a6938cb8750: Loading layer [=====>] 4.096kB/4.096kB
206e0a635703: Loading layer [=====>] 1.683GB/1.683GB
78990be6c067: Loading layer [=====>] 16.94MB/16.94MB
36c1ef6801b4: Loading layer [=====>] 173.6kB/173.6kB
41c377a7ace2: Loading layer [=====>] 173.1kB/173.1kB
007d0bb49307: Loading layer [=====>] 30.72kB/30.72kB
bdea22460106: Loading layer [=====>] 275.7MB/275.7MB
48acf3839ace: Loading layer [=====>] 5.632kB/5.632kB
301fc91b48ca: Loading layer [=====>] 52.45MB/52.45MB
43f2a987f849: Loading layer [=====>] 3.072kB/3.072kB
c91ae9930a72: Loading layer [=====>] 4.238MB/4.238MB
917ad535c416: Loading layer [=====>] 1.724MB/1.724MB
f7c7873042ab: Loading layer [=====>] 11.28MB/11.28MB
5574f969381e: Loading layer [=====>] 369.8MB/369.8MB
9ad95af43f47: Loading layer [=====>] 407.5MB/407.5MB
56b59f8eb593: Loading layer [=====>] 5.632kB/5.632kB
Loaded image: ccr.ccs.tencentyun.com/hcm/server.release:121736
[root@16-300g-0001 ~]#
```

4. 登录到master服务器上执行镜像更新脚本（脚本一般是在/root目录下）

`sh update_hcm.sh server版本号 client版本号`

```
[root@8-16-300g-0001 ~]# sh update_hcm.sh 121736 121725
deployment.apps/hcm-cloud image updated
deployment.apps/hcm-script image updated
deployment.apps/hcm-core image updated
deployment.apps/celery-beat image updated
deployment.apps/celery-quick image updated
deployment.apps/celery-slow image updated
Error from server (NotFound): deployments.apps "celery-syn" not found
deployment.apps/celery-salary image updated
Error from server (NotFound): deployments.apps "celery-time" not found
Error from server (NotFound): deployments.apps "celery-shortcut" not found
deployment.apps/celery-monitor image updated
Error from server (NotFound): deployments.apps "celery-default" not found
update success!!!
[root@8-16-300g-0001 ~]#
```

5. 查看更新进度

- (1) `kubectl get po` (#检查pod是否全部是running状态, master节点执行即可)
- (2) 浏览器访问`http(s)://登录地址/server/version` `http(s)://登录地址/client/version` 前后端版本查看版本是否已经为最新的版本
- (3) `kubectl logs -f | app=hcm-script --tail=200` (master上执行查看日志是否打印了end_update120)